

DEVOPS

RESOURCES	2
Some Resume Ready DevOps projects (Part-I)	2
Some Resume Ready DevOps projects (Part-II)	3
Some Resume Ready DevOps projects.(Part-III)	4
Real-world Prometheus Deployment: A Practical Guide for Kubernetes Monitoring	4
DevOps With AWS	5
Course Details.....	5
AWS Certifications	5
Client Server Architecture	5
1-Tier Architecture (Single Layer)	7
2-Tier Architecture	8
3-Tier Architecture	10
Network Infrastructure and DNS	13
HTTP & HTTPS Concepts.....	15
OSI Model (Open Systems Interconnection)	17
AWS Services Overview	18
AWS Overview and Shared Responsibility Model.....	18
Elasticity	20
High Availability (HA)	21
AWS Global Infrastructure - Regions and Availability Zones (AZs).....	22
EC2, Load Balancer, Elastic Beanstalk & LightSail	24
AWS Lambda for EC2 Automation.....	26
AWS Elastic Block Storage (EBS)	28
AWS S3 (Simple Storage Service).....	30

RESOURCES

SOME RESUME READY DEVOPS PROJECTS (PART-I)

Jenkins CICD with GitHub Integration

 <https://lnkd.in/gbDBEASe>

Deploy Java application with Complete CI/CD pipeline Jenkins

 <https://lnkd.in/gSJaf4VN>

DevOps CICD Pipeline Setup Through { Jenkins | Nexus | Sonarqube | Tomcat | Maven }

 <https://lnkd.in/gvDYKMxV>

Jenkins CICD with GitHub Integration

 https://lnkd.in/g-eaM_EA

Continuous Delivery Pipeline for Amazon ECS Using Jenkins, GitHub, and Amazon ECR

 <https://lnkd.in/gN7WXAnX>

Provisioning 2-tiers and 3-tiers AWS VPC infrastructure using Terraform

 https://lnkd.in/g2Y_Nk6N

Best Jenkins Pipeline Tutorial For Beginners

 <https://lnkd.in/gF93WwrB>

End to End CI/CD pipeline using GitHub Actions for Android Application

 <https://lnkd.in/gB-Vvi7r>

Create, publish, and store Nginx Docker image in AWS S3 using Docker and Bash

 <https://lnkd.in/g4iUncv9>

SOME RESUME READY DEVOPS PROJECTS (PART-II)

➔ Real-Time CI CD Pipeline From Scratch with Jenkins:-

🔗 <https://lnkd.in/day6k4GE>

➔ Creating AWS EKS Cluster using Terraform:-

🔗 <https://lnkd.in/gjtjHuw4>

➔ Cost Optimization through AWS Lambda Function:-

🔗 <https://lnkd.in/drDazJ9E>

➔ AWS Ninja — Convert EBS Volume from gp2 to gp3:-

🔗 <https://lnkd.in/d6mUFdEK>

➔ Real-World CI/CD DevSecOps Pipeline for Deployment of Python Web-app:-

🔗 <https://lnkd.in/d-TJisDn>

➔ AWS S3 Event Triggering Shell Script Used by Netflix, Airbnb, Adobe, Expedia, and Others :-

🔗 <https://lnkd.in/dXhzdwP8>

➔ Deploy Multi Stage CICD Pipeline with Amazon EKS :-

🔗 <https://lnkd.in/dEn7Vexj>

➔ Deploying Spring Boot application on K8S :-

🔗 https://lnkd.in/d8E_N_eg

➔ Netflix Clone application via K8's:-

🔗 <https://lnkd.in/d2zBUdMG>

🔗 <https://lnkd.in/dPVTDK9r>

➡ Deploy a Flask Application to a EKS Cluster using Jenkins and ArgoCD :-

🔗 <https://lnkd.in/dn8WtnWj>

SOME RESUME READY DEVOPS PROJECTS.(PART-III)

➡ Deploying Super Mario on Amazon EKS

🔗 (<https://lnkd.in/d2JVCw6M>)

➡ 🚀 Designing a 10-Microservices Application Deployment on EKS! 🤖

🔗 (<https://lnkd.in/dFuq9mMA>)

➡ Uber Clone DevSecOps Project

🔗 (https://lnkd.in/dTXm9_h9)

➡ 3-Tier Robot Shop App On EKS

🔗 (<https://lnkd.in/dtFK8Xj7>)

➡ Full Stack CICD Virtual Browser Project

🔗 (<https://lnkd.in/d2ZErR2b>)

REAL-WORLD PROMETHEUS DEPLOYMENT: A PRACTICAL GUIDE FOR KUBERNETES MONITORING

🔗 Source Link: <https://github.com/NotHarshhaa/Learning-Prometheus>

🔗 Blog Link: <https://blog.prodevopsguy.xyz/real-world-prometheus-deployment-a-practical-guide-for-kubernetes-monitoring>

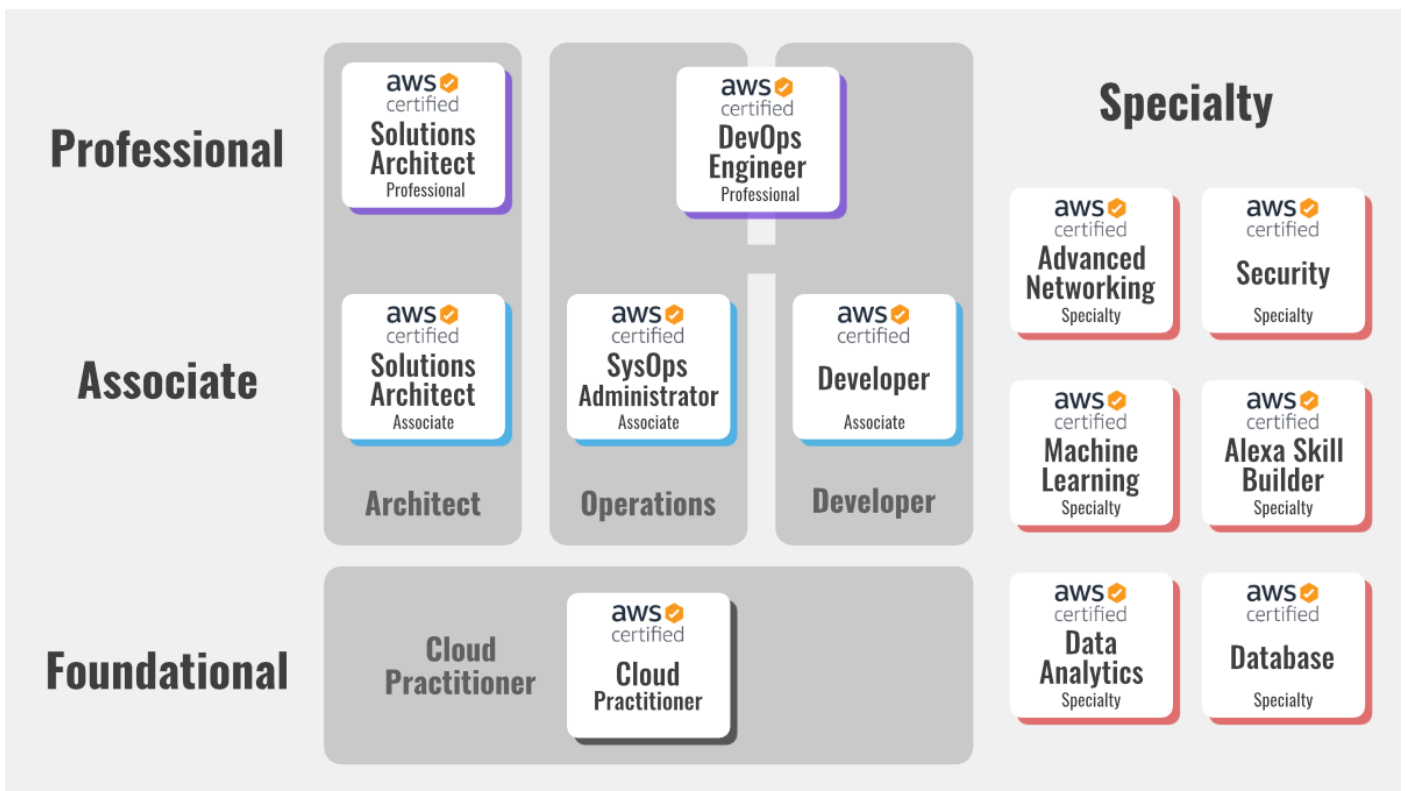
DEVOPS WITH AWS

COURSE DETAILS

Section	Details
Course Strategy	
Part 1: AWS - Manual	Focus on understanding and manually working with AWS core services.
Part 2: DevOps - Automation	Learn DevOps tools and practices for automating workflows and deployments.
Part 3: AWS with DevOps	Combine AWS services with DevOps practices for end-to-end cloud solutions.

AWS CERTIFICATIONS

Level	AWS Certifications	Description
Foundational	Cloud Practitioner	Covers AWS basics and is designed for individuals starting their cloud journey.
Associate	Developer Associate	Focuses on using AWS for application development and maintenance.
	SysOps Admin Associate	Centers on deploying, managing, and operating scalable and fault-tolerant systems on AWS.
	Solutions Architect Associate	Targets those designing distributed systems on AWS.
Professional	DevOps Engineer	Combines advanced AWS skills with DevOps principles for managing and automating infrastructure.
	Solutions Architect Professional	Focuses on complex AWS architectures and advanced solution designs.



CLIENT SERVER ARCHITECTURE

Fundamentals

- **Resource:** Information or data that a client requests from a server.

- **Client:** A device or program that requests resources or services from a server. (e.g., Web browser, email client)
- **Server:** A device or program that provides resources or services to clients. (e.g., Web server, email server)
- **Network:** The communication pathway that connects clients and servers. (e.g., Internet, local network)

Key Concepts

- **Request-Response Model:**
 - Clients initiate communication by sending requests to the server.
 - Servers process requests and send back responses to the client.
- **Role Flexibility:** Devices can act as both a client and a server depending on the situation.

Example: Web Browsing

1. You type "google.com" into your web browser (client).
2. The browser sends a request over the internet to the Google server.
3. The Google server receives the request and processes it.
4. The server sends the requested webpage (resource) back to your browser.
5. Your browser displays the webpage.

Advantages of Client-Server Architecture

- **Centralized Data Management:** Data is stored and managed on the server, making it easier to control access, maintain data integrity, and perform backups.
- **Scalability:** Servers can be upgraded or more servers can be added to handle increasing client demands.
- **Security:** Servers can implement security measures to protect data from unauthorized access.
- **Flexibility:** Clients and servers can be different types of devices running different operating systems.

Disadvantages of Client-Server Architecture

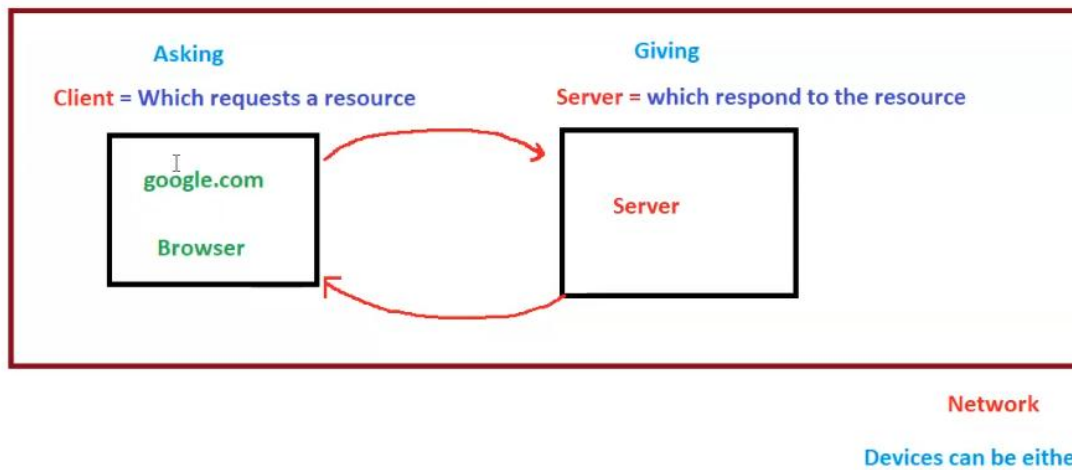
- **Single Point of Failure:** If the server fails, the entire system can be affected.
- **Network Dependency:** The system relies on a stable network connection.
- **Cost:** Servers can be expensive to purchase and maintain.

Additional Notes:

- Client-server architecture is a fundamental concept in networking and distributed computing.
- Many applications and services we use daily rely on this model (e.g., the web, email, online gaming).

Client Server Architecture

Resource = Information



1-TIER ARCHITECTURE (SINGLE LAYER)

Fundamentals

- **Single Layer:** All components (presentation, business logic, and data storage) reside on a single machine, often the user's local machine or a standalone server.
- **Localhost:** A hostname that refers to the current computer on which the code is running.
- **Client/Server on Same Machine:** The application acts as both the client and the server, running on the same system.
- **Limited Scalability:** Difficult to scale as all processing occurs on a single machine.
- **Suitable for Simple Applications:** Ideal for basic applications with low user traffic and minimal resource requirements.

Example: Local Development Environment

- A developer (like Ganesh in the diagram) builds and tests a website on their laptop.
- The website files, database, and browser all reside on the same machine.
- The developer accesses the website using "localhost" in the browser's address bar.

Advantages of 1-Tier Architecture

- **Simplicity:** Easy to set up and maintain due to its single-layer structure.
- **Cost-Effective:** No need for separate servers or network infrastructure.
- **Suitable for Offline Use:** Can function without a network connection.

Disadvantages of 1-Tier Architecture

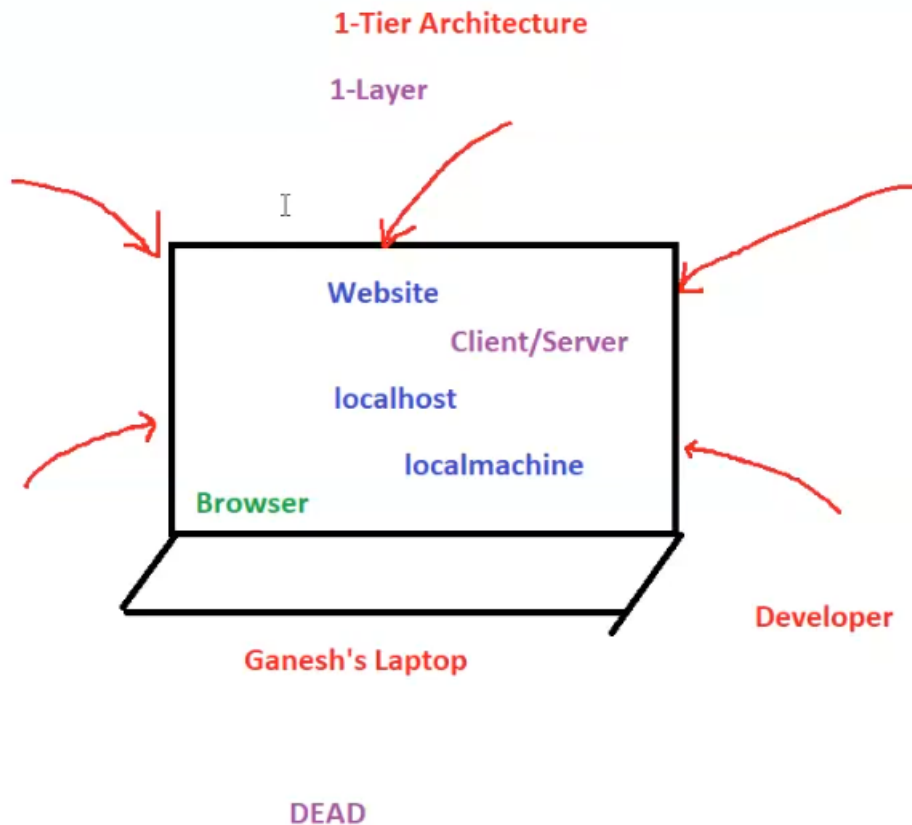
- **Limited Scalability:** Cannot handle large numbers of users or complex applications.
- **Security Concerns:** Data might be more vulnerable as it resides on the same machine as the client.
- **Performance Bottlenecks:** All processing occurs on a single machine, which can lead to performance issues.

Additional Notes

- 1-tier architecture is often used for personal projects, small-scale applications, or during the initial development phase of larger projects.
- It is less common in production environments due to its limitations.

DEAD

- The "DEAD" label likely signifies that this architecture is not suitable for large-scale or complex applications due to its limitations in scalability and performance.



2-TIER ARCHITECTURE

Fundamentals

- **Two Layers:**
 - **Client Layer:** Handles user interface and user interactions.
 - **Server Layer:** Houses the application logic and the database.
- **Direct Communication:** Clients communicate directly with the server.
- **Suitable for Moderate-Scale Applications:** Can handle more users and complexity than 1-tier but has limitations compared to 3-tier.

Components

- **Clients:** Various devices (e.g., computers, mobiles) running client applications.
- **Server:** A dedicated machine running the application and database.

- **Application:** Processes client requests, performs business logic, and interacts with the database. (e.g., written in Java, .NET, Python)
- **Database:** Stores and manages application data. (e.g., MySQL, Oracle, MS SQL)

Advantages of 2-Tier Architecture

- **Improved Performance:** Compared to 1-tier, separate server handles processing, potentially improving performance.
- **Easier Development:** Simpler to develop and maintain than more complex architectures.
- **Cost-Effective:** Less expensive than 3-tier as it requires fewer servers.

Disadvantages of 2-Tier Architecture

- **Scalability Challenges:** Can become a bottleneck as the number of users increases.
- **Security Concerns:** Direct client-server communication can pose security risks.
- **Maintenance Overhead:** Updates and modifications can be more complex as the application and database are tightly coupled.

"BIG NO FOR PROD"

- The image emphasizes that 2-tier architecture is generally not recommended for production environments, especially for large-scale or critical applications.
- This is due to its limitations in scalability, security, and maintainability.

Potential Issues Illustrated

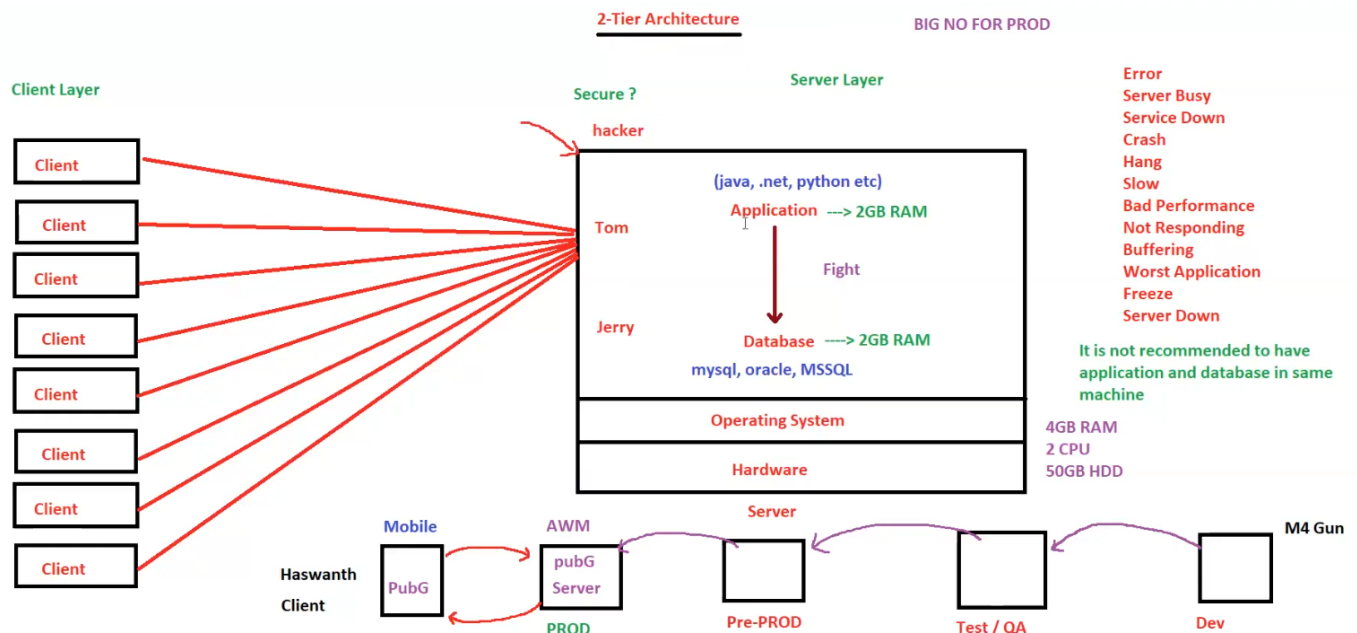
- **Server Overload:** Multiple clients accessing the server simultaneously can lead to errors, slowdowns, crashes, and poor performance.
- **Security Vulnerability:** A "hacker" is shown, highlighting the potential security risks of direct client-server communication.
- **Resource Constraints:** The server having limited resources (2GB RAM) for both the application and database can further impact performance.

Example: "Tom and Jerry" Analogy

- "Tom" (application) and "Jerry" (database) constantly fighting represents the potential conflicts and performance issues when both reside on the same server with limited resources.

Additional Notes

- 2-tier architecture was more common in the early days of client-server computing.
- While not ideal for large production systems, it can still be suitable for certain applications with moderate user loads and less demanding requirements.



3-TIER ARCHITECTURE

3-Tier Architecture

Fundamentals

- **Three Layers:**
 - **Client Layer:** Presentation tier (user interface) - interacts with the user.
 - **Application Layer:** Logic tier (business logic, application code) - processes data and handles business rules.
 - **Database Layer:** Data tier (data storage) - stores and manages data.
- **Indirect Communication:** Clients do not directly access the database; they interact with the application server.
- **Enhanced Scalability and Security:** Each tier can be scaled independently, and the database is protected behind the application server.
- **Suitable for Complex Applications:** Ideal for large-scale applications with high user traffic and complex requirements.

Components

- **Clients:** Devices like web browsers accessing the application.
- **Web Server:** Handles client requests and redirects them to the application server (e.g., Nginx, Apache, httpd).
- **Application Server:** Hosts the application logic and processes client requests (e.g., Tomcat, IIS).
- **Database Server:** Stores and manages the application's data (e.g., MySQL).

Communication Flow

1. Client sends a request to the web server.
2. Web server redirects the request to the application server.
3. Application server processes the request, interacts with the database server if needed, and sends the response back to the web server.
4. Web server forwards the response to the client.

Network Security

- **Public Network:** The web server is accessible from the public internet.
- **Private Network:** The application server and database server are in a private network, protected from direct access from the internet. This enhances security.

N-Tier Architecture

- **Extends 3-Tier:** Can have more than three tiers, with additional layers for tasks like security, caching, load balancing, etc.
- **Increased Flexibility:** Offers greater flexibility and scalability for complex applications.

Advantages of 3-Tier/N-Tier Architecture

- **Scalability:** Each tier can be scaled independently to handle increased load.
- **Maintainability:** Easier to maintain and update as changes in one tier don't necessarily affect others.
- **Security:** Database is protected behind the application server.
- **Reusability:** Components can be reused across multiple applications.

Disadvantages of 3-Tier/N-Tier Architecture

- **Complexity:** More complex to design and implement than 2-tier.
- **Performance:** Can be slower than 2-tier if not designed and optimized properly.
- **Cost:** May require more hardware and resources.

Additional Notes

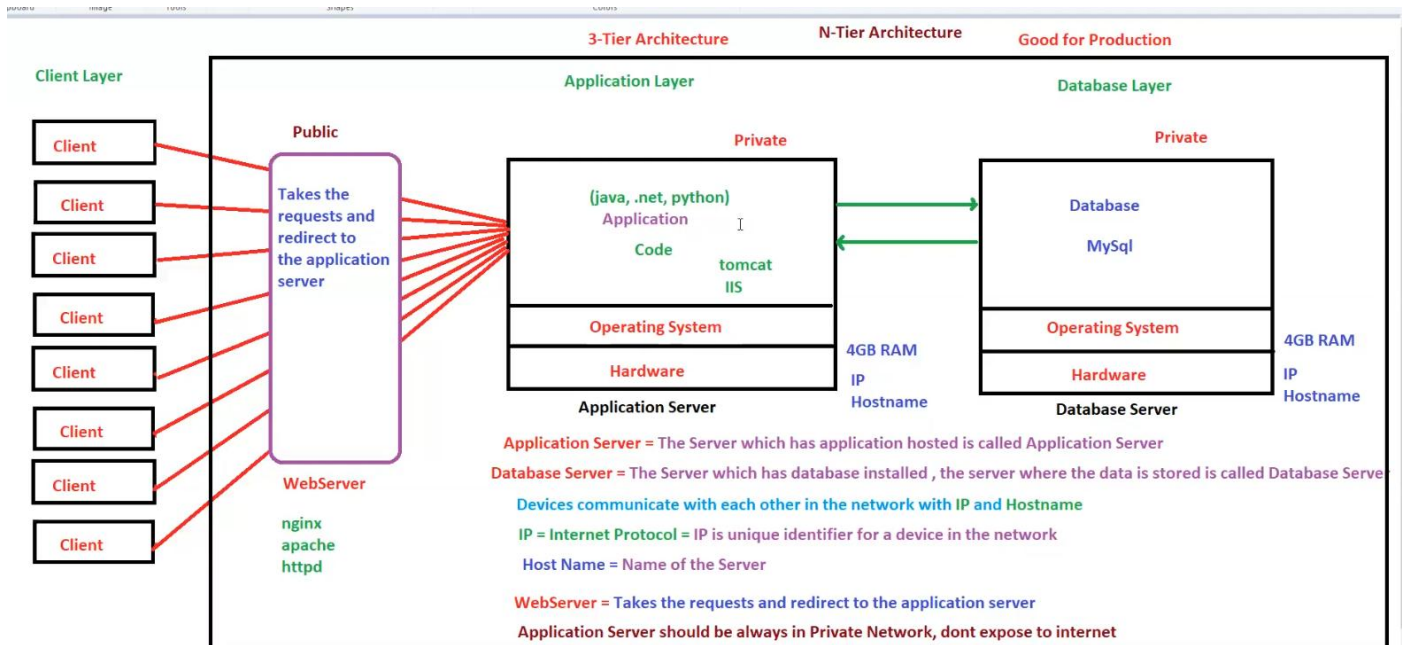
- 3-tier and N-tier architectures are widely used for enterprise-level applications and web services.
- They provide a robust and scalable solution for handling complex applications and large user bases.

Diagram Explanation

The diagram illustrates the different tiers, components, and communication flow in a 3-tier architecture. It also highlights the importance of keeping the application server and database server in a private network for security reasons.

Key Terms

- **IP Address:** A unique identifier for a device on a network.
- **Hostname:** A human-readable name for a server.



Tier 1 vs Tier 2 vs Tier 3

Feature	1-Tier Architecture	2-Tier Architecture	3-Tier Architecture
Layers	Single	Two (Client, Server)	Three (Client, Application, Database)
Components	All components on one machine	Client and server	Client, web server, application server, database server
Communication	N/A (all on one machine)	Direct client-server	Client <-> Web Server <-> Application Server <-> Database Server
Scalability	Very limited	Moderate	High
Security	Low	Moderate	High
Complexity	Simple	Moderate	Complex
Performance	Limited by single machine	Can be improved over 1-tier	Potentially high with proper optimization
Cost	Low	Moderate	High
Typical Use Cases	- Personal projects - Small-scale applications - Local development	- Applications with moderate user load	- Enterprise applications - High-traffic web services - Complex systems
Suitable for Production?	No	Generally, not recommended	Yes
Advantages	- Simplicity - Cost-effectiveness	- Improved performance over 1-tier - Easier development than 3-tier	- High scalability - Maintainability - Security - Reusability
Disadvantages	- Limited scalability - Security concerns - Performance bottlenecks	- Scalability challenges - Security concerns - Maintenance overhead	- Complexity - Potential performance overhead - Higher cost

1. Domain Name System (DNS)

- **Function:** Translates human-readable domain names (e.g., google.com) into machine-readable IP addresses.
- **Hierarchy:**
 - **Root Name Servers (RNS):** The top of the hierarchy, responsible for delegating to Top-Level Domain (TLD) servers.
 - **Top-Level Domains (TLDs):** .com, .org, .in, etc.
 - **Name Servers:** Authoritative servers for specific domains, holding the actual IP address records (A records).
 - **Local DNS:** Cached records on a local network (e.g., your ISP's DNS server or a DNS server within your organization).

2. Network Components

- **Firewall:** A security device that controls network traffic based on rules (allow/deny), protecting against unauthorized access.
- **Load Balancer (LB):** Distributes incoming network traffic across multiple servers (e.g., web servers) to ensure high availability and performance.

3. Web Communication

- **Browser:** The client software used to access websites.
- **Web Server:** Hosts the website's files and serves them to users.
- **Application Server:** Handles the logic and processing of web applications.
- **Database Server:** Stores the data used by the application.

4. Protocols

- **HTTP (Port 80):** Unsecured protocol for transferring web content.
- **HTTPS (Port 443):** Secure protocol using SSL/TLS encryption for safe communication.
- **SSH (Port 22):** Secure protocol for remote access to Linux machines.
- **RDP (Port 3389):** Protocol for remote access to Windows machines.

5. Network Packet Flow

- **Browser:** Sends a DNS request for the hostname (e.g., google.com) to the Local DNS.
- **Local DNS:** Checks its cache. If not found, forwards the request up the DNS hierarchy.
- **DNS Resolution:** The request travels up to the Root Name Servers, then to the TLD server, and finally to the authoritative Name Server for the domain, which returns the IP address.
- **Browser:** Receives the IP address and sends an HTTP or HTTPS request to the web server at that address.
- **Firewall:** Checks the request against its rules.
- **Load Balancer:** If present, directs the request to an appropriate web server.

- **Web Server:** Responds with the requested web page.

6. Other Concepts

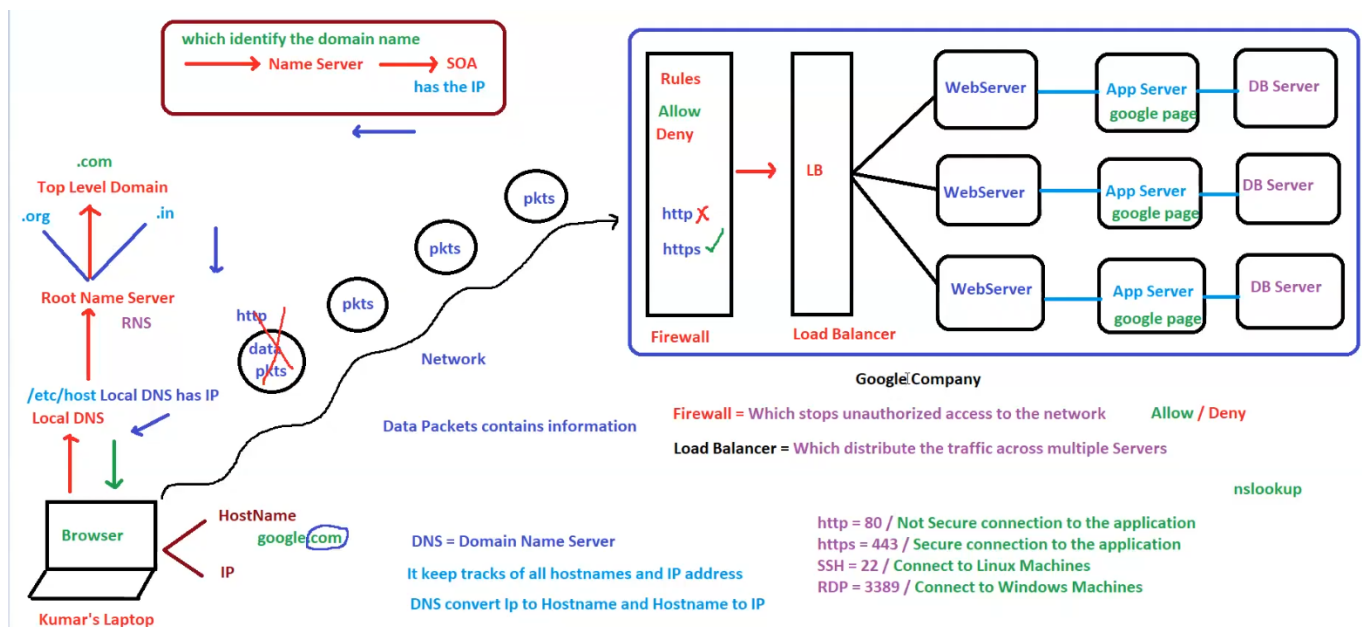
- **/etc/hosts:** A local file on a computer that can be used to override DNS mappings.
- **nslookup:** A command-line tool used to query DNS servers.

Remember:

- DNS is essential for navigating the internet.
- Firewalls and Load Balancers are critical components for network security and performance.
- Understanding protocols like HTTP, HTTPS, SSH, and RDP is important for secure communication.

Additional Notes:

- The diagram shows a simplified network architecture. Real-world networks can be much more complex.
- The "Data Packets contain information" note highlights that network traffic carries the actual data being exchanged.



HTTP & HTTPS Concepts

- **HTTP (HyperText Transfer Protocol)**
 - Default port: **80**
 - Used to transfer data between browsers and servers.
 - **Not secure** as data is transferred in **plain text**.
- **HTTPS (HyperText Transfer Protocol Secure)**
 - Default port: **443**
 - Uses **SSL/TLS certificates** for encryption.
 - **Secure** as data is encrypted.

HTTP Status Codes

- **404** – Page not found
- **500** – Internal Server Error
- **503** – Service Unavailable
- **200** – Page Found / Success

Protocols & Ports

- **Protocol Format:** protocol://domainname:portnumber
- **Examples:**
 - http://192.168.10.20:80
 - http://192.168.10.20:8080
 - http://192.168.10.20
- **Port Customization:**
 - Ports can be customized at the application level.
 - For customers, standard ports (**80 or 443**) should be used.

Transmission Control Protocol (TCP)

- Establishes connection between two hosts.
- Works as a **bridge** for data transfer.
- More **reliable** than UDP.

User Datagram Protocol (UDP)

- Used for faster data transmission.

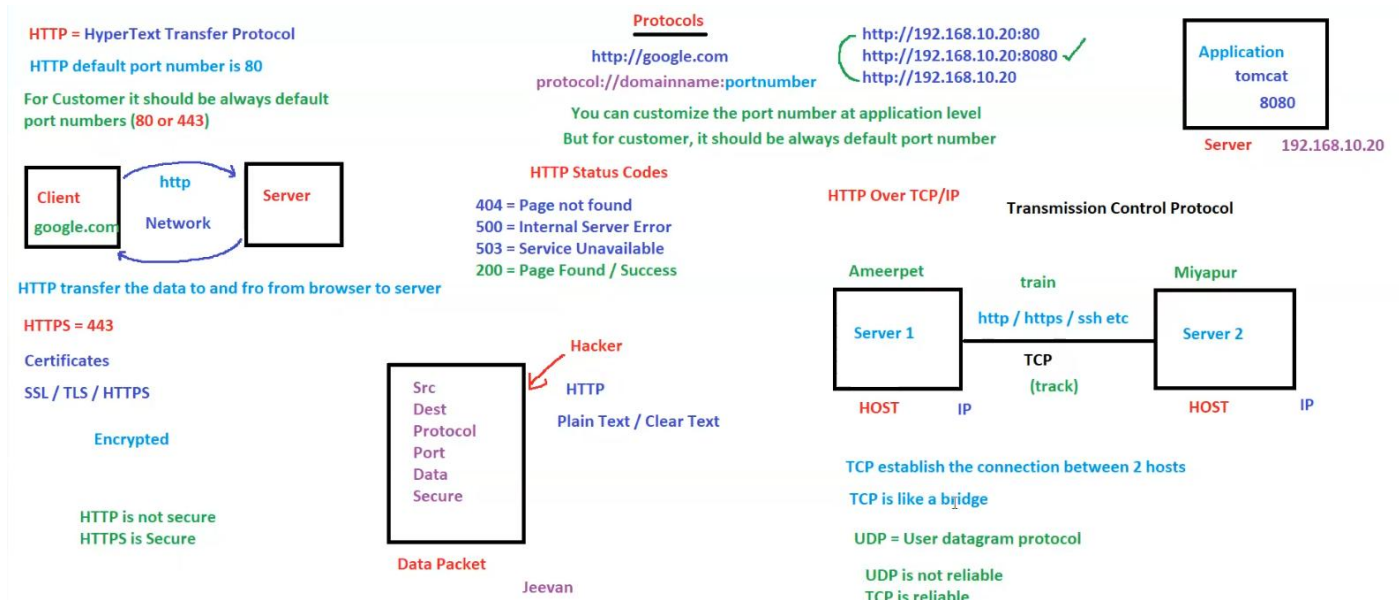
- **Not reliable** compared to TCP.

Network Communication

- **Client (google.com) ↔ Server** via HTTP/HTTPS.
- **HTTP over TCP/IP** ensures communication between servers.
- **Example:**
 - Server 1 (Ameerpet) ↔ Server 2 (Miyapur) using TCP.

Security Concerns

- **HTTP is vulnerable to hacking** as data is transferred in **plain text**.
- **Hackers can intercept data packets.**
- **HTTPS ensures encryption** and secures data.



OSI Model (Open Systems Interconnection)

- The OSI model consists of **7 layers**, each responsible for a specific aspect of network communication.

7 Layers of OSI Model

1. Application Layer

- Provides network services to end-users.
- Examples: **HTTP, HTTPS**

2. Presentation Layer

- Ensures data is in a readable format.
- Examples: **SSH, RDP**

3. Session Layer

- Manages and controls connections between computers.

4. Transport Layer

- Ensures reliable data transfer between devices.
- Examples: **TCP, UDP**

5. Network Layer

- Handles addressing and routing of data.
- Example: **IP (Internet Protocol)**

6. Data Link Layer

- Manages data transfer between directly connected nodes.

7. Physical Layer

- Concerned with the physical transmission of data (cables, signals, etc.).

Network Communication Example

- **Host 1 (Boy) ↔ Host 2 (Girl)**
 - Connected through **TCP or UDP** for communication.
 - This represents two devices communicating over a network.

Key Protocols

- **TCP (Transmission Control Protocol)**

- **UDP (User Datagram Protocol)**



7 Layers

- Application http, https
- Presentation SSH, RDP etc
- Session
- Transport (TCP) I
- Network (IP)
- Data Link
- Physical

AWS is a Group of Services

We can access AWS Services through Amazon Management Console

VM's = Instances

EC2 = Elastic Compute Cloud java, .net, python

EC2 is a AWS Service where we can create Virtual Machines

EC2 is a AWS Service
where we can launch EC2
instances

Shared Responsibility Model

Customer Responsibility

Customer

Provider

Provider

.zip
java, .net, python
Application



Sai
Client

3 T's

- Elasticity
- Scalability
- High Availability



AWS

AWS doesn't have any access inside your VM's
ElasticBeanStalk = Easy and Quick deployment of applications in AWS

PAAS
Azure

SAAS

Salesforce

Gmail, WebEx, Zoom etc

AWS Overview and Shared Responsibility Model

1. AWS Basics

- **AWS (Amazon Web Services)** is a group of cloud services.
- We can access AWS services via the **Amazon Management Console**.
- **VMs (Virtual Machines) = Instances** in AWS.

2. EC2 (Elastic Compute Cloud)

- **EC2** is an AWS service that allows users to create and launch virtual machines.
- **EC2 falls under the Infrastructure as a Service (IAAS) model.**
- AWS does not have access inside your virtual machines.

3. Shared Responsibility Model

- **Customer Responsibility:** Application and Data management.
- **Provider Responsibility:** Network, Data Center, and Virtualization.
- **Shared Responsibility:**
 - Customers do not need to worry about servers.
 - Customers have no control over the physical servers.

4. Cloud Service Models

1. IAAS (Infrastructure as a Service)

- Example: **AWS EC2**
- Customers manage applications, data, and OS, while AWS handles virtualization and infrastructure.

2. PAAS (Platform as a Service)

- Example: **Azure, AWS Elastic Beanstalk**
- Provides an environment for application development and deployment.

3. SAAS (Software as a Service)

- Examples: **Salesforce, Gmail, WebEx, Zoom**
- Fully managed applications accessible via the internet.

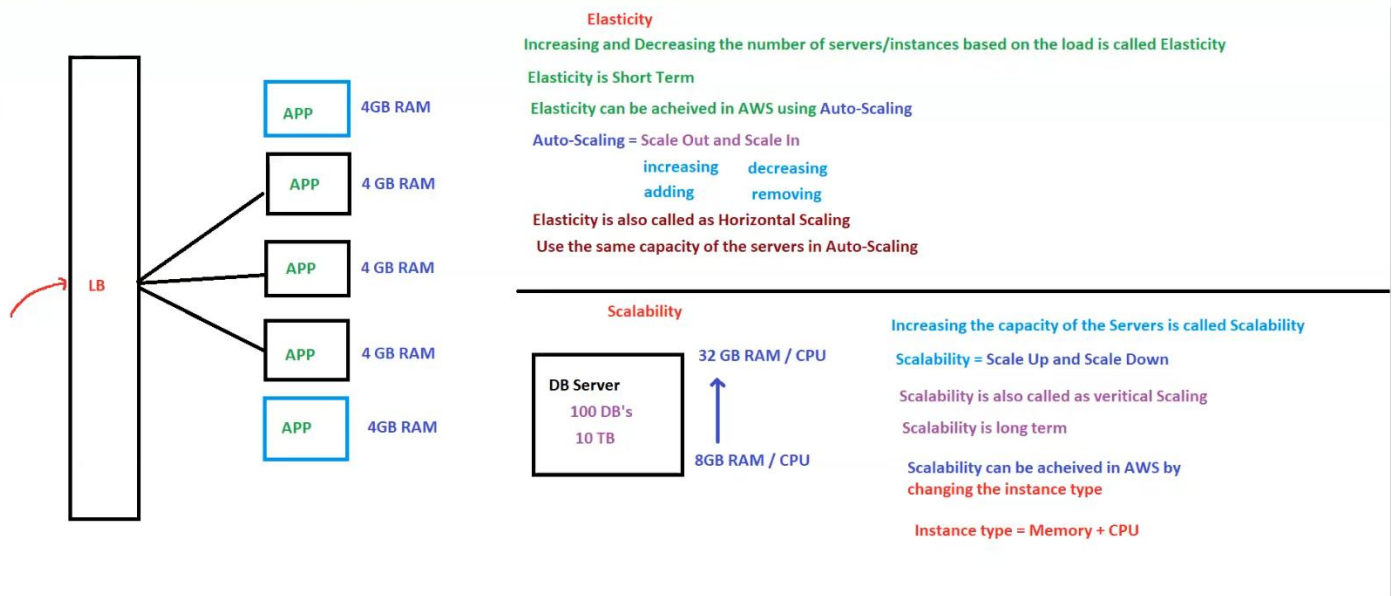
5. Benefits of AWS (The 3 T's)

- **Elasticity**
- **Scalability**
- **High Availability**

6. AWS Infrastructure

- AWS physical host machines run multiple virtual machines (VMs) using a **Hypervisor**.
- **Elastic Beanstalk** is an AWS service for easy and quick application deployment.

ELASTICITY



Elasticity

- **Definition:** Increasing and decreasing the number of servers/instances based on the load is called **Elasticity**.
- **Characteristics:**
 - Elasticity is **short-term**.
 - Achieved in **AWS** using **Auto-Scaling**.
 - **Auto-Scaling** involves:
 - **Scale Out** (increasing/adding instances)
 - **Scale In** (decreasing/removing instances)
 - Also known as **Horizontal Scaling**.
 - Uses **same capacity servers** in Auto-Scaling.

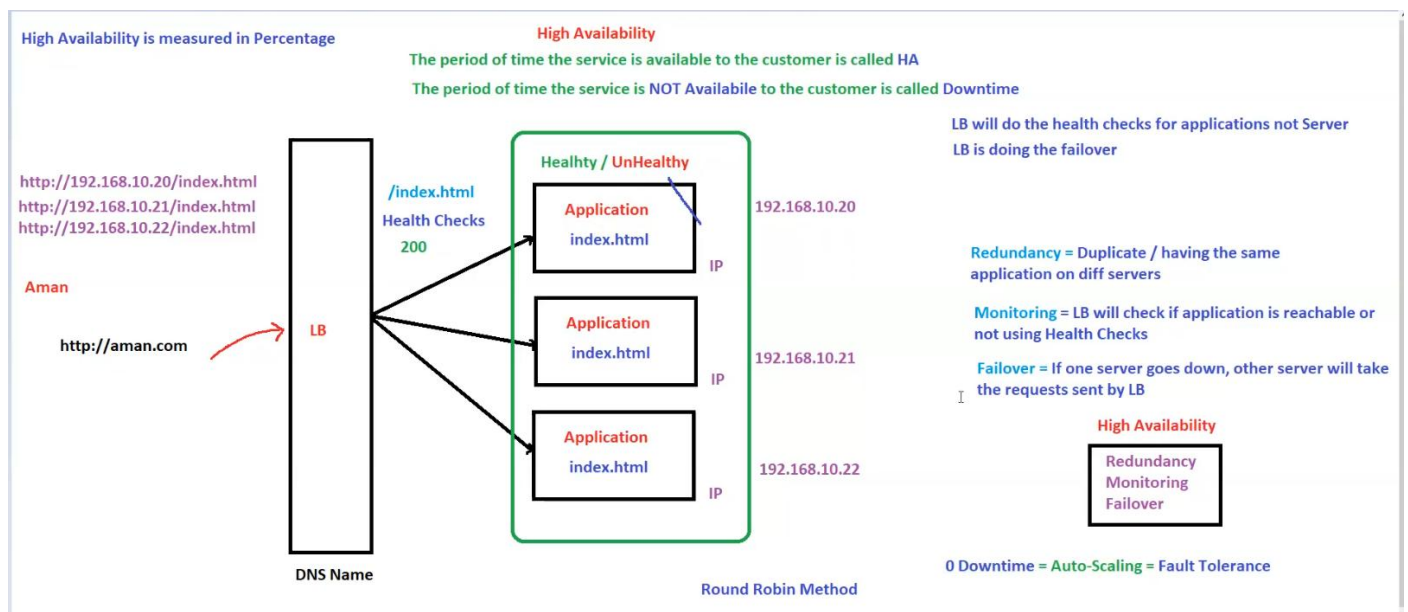
Scalability

- **Definition:** Increasing the **capacity** of servers is called **Scalability**.
- **Characteristics:**
 - Scalability = **Scale Up and Scale Down**.
 - Also called **Vertical Scaling**.
 - Scalability is **long-term**.
 - Achieved in **AWS** by **changing the instance type**.
 - **Instance Type** = Memory + CPU.

Key Differences

Feature	Elasticity (Horizontal Scaling)	Scalability (Vertical Scaling)
Definition	Adjusting the number of instances	Increasing server capacity
Timeframe	Short-term	Long-term
AWS Feature	Auto-Scaling	Changing Instance Type
Process	Scale In/Out (adding/removing instances)	Scale Up/Down (increasing/decreasing resources)
Capacity	Uses same capacity servers	Increases capacity of the same server

HIGH AVAILABILITY (HA)



High Availability (HA)

- **Definition:** The period of time the service is **available** to the customer is called **High Availability (HA)**.
- **Downtime:** The period of time the service is **not available** to the customer is called **Downtime**.
- HA is measured in percentage.

Key Components of High Availability

1. **Redundancy:**
 - **Definition:** Having duplicate instances of the same application on different servers.
2. **Monitoring:**
 - **Definition:** Load Balancer (LB) checks if the application is reachable using **Health Checks**.

- Health Check: Verifies the availability of **index.html** and expects a **200 status**.

3. Failover:

- **Definition:** If one server goes down, other servers take over the requests handled by the Load Balancer.

Load Balancer (LB)

- Handles incoming traffic and distributes it across multiple servers.
- Performs **Health Checks** for applications (not servers).
- Performs **failover** to redirect traffic if a server goes down.
- Uses **Round Robin Method** for load distribution.

Ensuring 0 Downtime

- Auto-Scaling = Fault Tolerance
- Combining:
 - Redundancy
 - Monitoring
 - Failover
Leads to **High Availability**.

Example Setup

- **DNS Name:** http://aman.com
- **Backend servers with IPs:**
 - 192.168.10.20
 - 192.168.10.21
 - 192.168.10.22
- Each server hosts index.html, and the Load Balancer distributes traffic among them.

AWS GLOBAL INFRASTRUCTURE - REGIONS AND AVAILABILITY ZONES (AZS)

AWS Global Infrastructure - Regions and Availability Zones (AZs)

1. AWS Global Infrastructure Overview

- **Region:** A physical location where AWS has infrastructure (e.g., Mumbai).
- **Availability Zone (AZ):** A **data center** within a region.
- A **Region** has multiple **AZs**, and each **AZ** is a group of **data centers** (DCs).

- Servers (EC2 Instances) are placed within AZs.
-

2. AWS Region and Availability Zones

- **Example:** AWS Region **Mumbai** (ap-south-1) has **three AZs**:
 - ap-south-1a
 - ap-south-1b
 - ap-south-1c
 - AZs are managed by AWS and are **interconnected via a network**.
 - **Best Practice:** Distribute instances across **multiple AZs** for high availability.
-

3. Communication Between AZs & Regions

- AZs within a **region** can communicate with each other **by default**.
 - Regions do **NOT communicate by default**, but it can be enabled if required.
 - Low Latency (1 sec) is good, while high latency (5 sec) is bad.
-

4. Load Balancing & Traffic Distribution

- Load Balancer (LB):
 - Distributes traffic across **multiple EC2 instances** in different AZs.
 - LB is specific to a **region**, not an AZ.
 - Instances across **AZs** can share data if required.
-

5. High Availability & Fault Tolerance

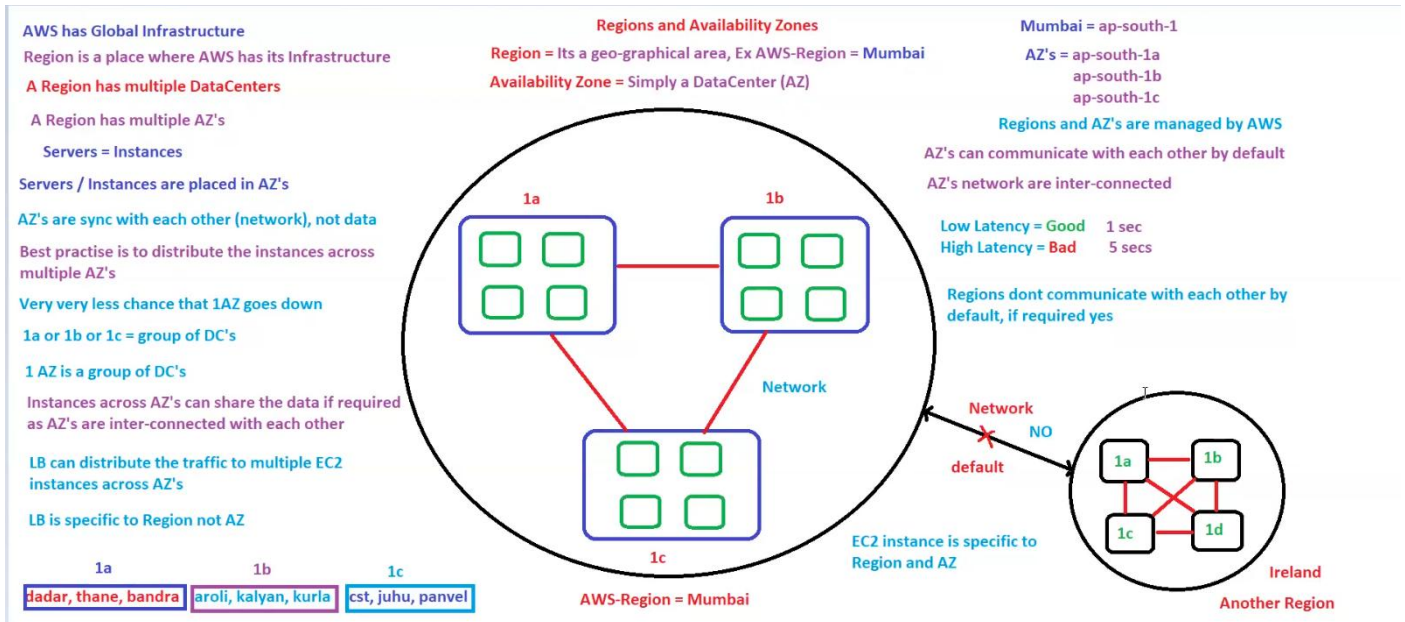
- Very low chance that an entire **AZ goes down**.
 - Distributing instances across **AZs** ensures **fault tolerance**.
 - If one AZ fails, others continue running.
-

6. Example Mapping (Mumbai Region)

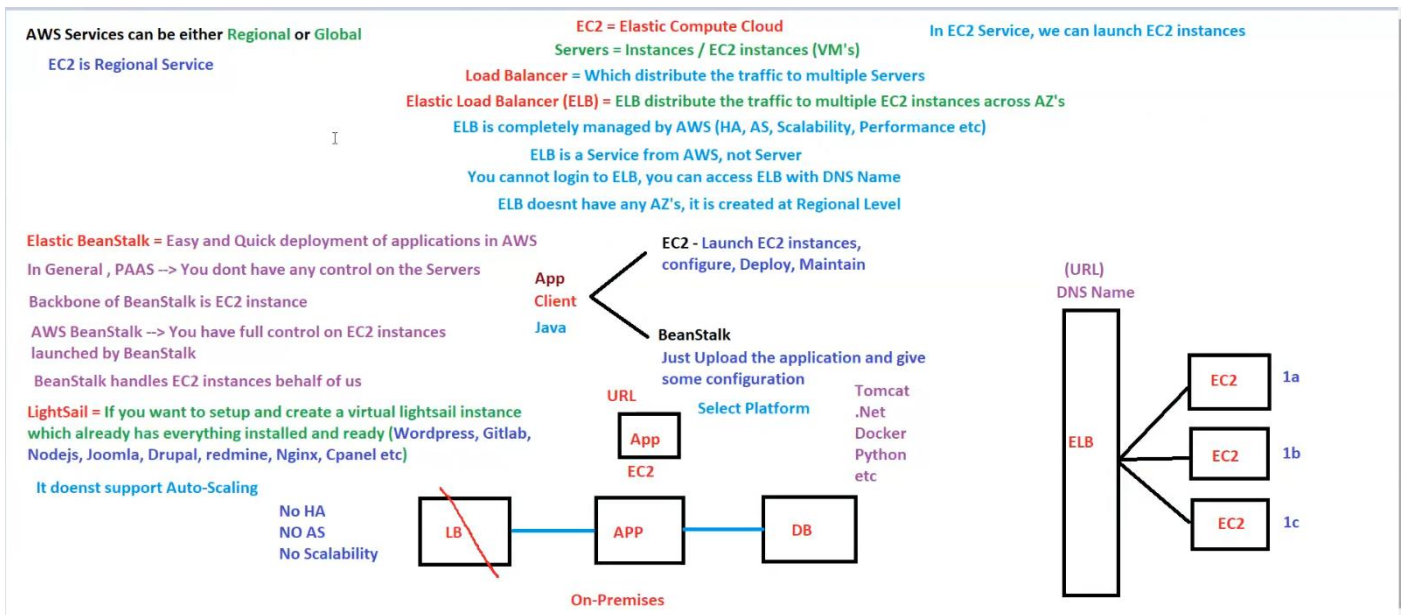
- **ap-south-1a:** Dadar, Thane, Bandra
 - **ap-south-1b:** Airoli, Kalyan, Kurla
 - **ap-south-1c:** CST, Juhu, Panvel
-

7. Summary

- ✓ **Region** = Group of AZs.
- ✓ **AZ** = Group of Data Centers (DCs).
- ✓ AZs within a region can communicate.
- ✓ Regions do NOT communicate by default.
- ✓ Load Balancers distribute traffic across AZs.
- ✓ **Best practice**: Use multiple AZs for high availability & redundancy.



EC2, LOAD BALANCER, ELASTIC BEANSTALK & LIGHTSAIL



1. AWS Services - Regional vs Global

- AWS Services can be either Regional or Global.
- EC2 (Elastic Compute Cloud) is a Regional Service
 - It means EC2 instances exist within a **specific AWS region**.

- EC2 instances are virtual machines (VMs) in the cloud.
-

2. EC2 Instances

- **EC2 = Elastic Compute Cloud** → Allows launching virtual servers (instances).
 - **Servers = Instances (EC2 VMs).**
 - EC2 instances need to be **configured, deployed, and maintained manually.**
-

3. Load Balancer (LB)

- **Distributes incoming traffic** across multiple EC2 instances.
 - **Elastic Load Balancer (ELB)**
 - ELB distributes traffic across EC2 instances in different AZs.
 - **Managed by AWS** (High Availability, Auto Scaling, Performance Optimization).
 - **Created at the Regional Level**, not tied to a specific AZ.
 - **Cannot log in to ELB**, only access it via DNS name.
-

4. Elastic Beanstalk (PaaS)

- **Simplifies application deployment** on AWS.
 - **Fully managed service**, handling EC2 instances, Load Balancing, and Auto Scaling.
 - **Key Features:**
 - Uses **EC2 instances as the backbone.**
 - You have **full control over EC2 instances.**
 - Just **upload your application** and provide configuration.
 - Supports **multiple platforms** like Tomcat, .Net, Docker, Python, etc.
-

5. AWS LightSail (Simplified VM Hosting)

- **Pre-configured cloud service** for hosting applications quickly.
 - Supports **WordPress, GitLab, Node.js, Joomla, Drupal, Nginx, Cpanel, etc.**
 - **Limitations:**
 - **✗ No Auto-Scaling.**
 - **✗ No High Availability (HA).**
 - **✗ No Scalability** (Cannot dynamically scale like EC2 with ELB).
-

6. EC2 Deployment Options

A. EC2 with ELB (Recommended)

- ✓ EC2 instances are deployed across **multiple AZs**.
- ✓ ELB **distributes traffic** to maintain high availability.
- ✓ **Auto Scaling** ensures instances scale up/down as needed.

B. EC2 with Beanstalk

- ✓ Easy deployment and management.
- ✓ **AWS manages scaling & load balancing**.
- ✓ **Ideal for developers** who don't want to manage infrastructure.

C. EC2 Without Load Balancer (Not Recommended)

- ✗ Single EC2 instance without redundancy.
- ✗ **No auto-scaling** = No ability to handle high traffic.
- ✗ **If the instance fails, the application is down.**

7. Summary

AWS Service	Use Case	Key Features
EC2	General-purpose compute	Manual setup, requires maintenance
ELB	Load balancing traffic	Auto-distributes requests across EC2 instances
Elastic Beanstalk	Easy application deployment	PaaS, fully managed, supports multiple platforms
LightSail	Quick app hosting	Simple VM setup, no scaling, no HA

- ✓ Use EC2 + ELB for **scalable and fault-tolerant architecture**.
- ✓ Use Beanstalk for **easy deployment with AWS-managed scaling**.
- ✓ Use LightSail for **simple, pre-configured environments without scaling needs**.

AWS LAMBDA FOR EC2 AUTOMATION

1. AWS Lambda Overview

- **Lambda is a Serverless service** → No need to manage servers.
- You can **run code** without provisioning infrastructure.
- Supports **multiple languages** like **Java, Python, .NET, Ruby, etc.**
- Lambda can be **triggered by events** (e.g., EC2 state changes, scheduled tasks, etc.).

2. Automating EC2 with Lambda

- **Lambda can perform EC2 operations:**
 - ✓ **Launch**
 - ✓ **Stop**

- ✓ Start
- ✓ Reboot
- ✓ Terminate/Delete

- Lambda is triggered based on:
 - EventBridge Rules (For Scheduled Events).
 - Launch Events (For EC2 instance events).
-

3. Event-Based EC2 Management Using Lambda

A. Scheduled Stop & Start of EC2 Instances

- 9 PM → STOP EC2 Instances
 - EventBridge Rule triggers STOPFn Lambda function.
 - Lambda stops all EC2 instances.
 - 6 AM → START EC2 Instances
 - EventBridge Rule triggers StartFn Lambda function.
 - Lambda starts all EC2 instances.
-

B. EC2 Termination Using Lambda

- Lambda can terminate EC2 instances using Python Code (TerminateFn).
 - This can be invoked manually or automatically via EventBridge.
 - Example Use Case: **Auto-terminate idle EC2 instances** to reduce costs.
-

4. AWS Services Involved

Service	Purpose
Lambda	Runs Python code to start/stop EC2 instances.
EventBridge	Schedules events (9 PM Stop, 6 AM Start).
EC2	Instances managed via Lambda automation.

5. Summary

- ✓ AWS Lambda is used for automating EC2 instance operations.
- ✓ EventBridge rules can trigger Lambda functions to start/stop EC2 instances on schedule.
- ✓ Lambda functions can be used for launching, stopping, and terminating EC2 instances automatically.
- ✓ Serverless automation reduces manual effort & optimizes cloud costs.

Lambda is used for Automations

Lambda
Lambda is Serverless
You can run the code without Server

Create Function

Lambda Function java, python, .net, ruby etc



Sajed

Launch
Stop
Start
Reboot
Terminate → Delete

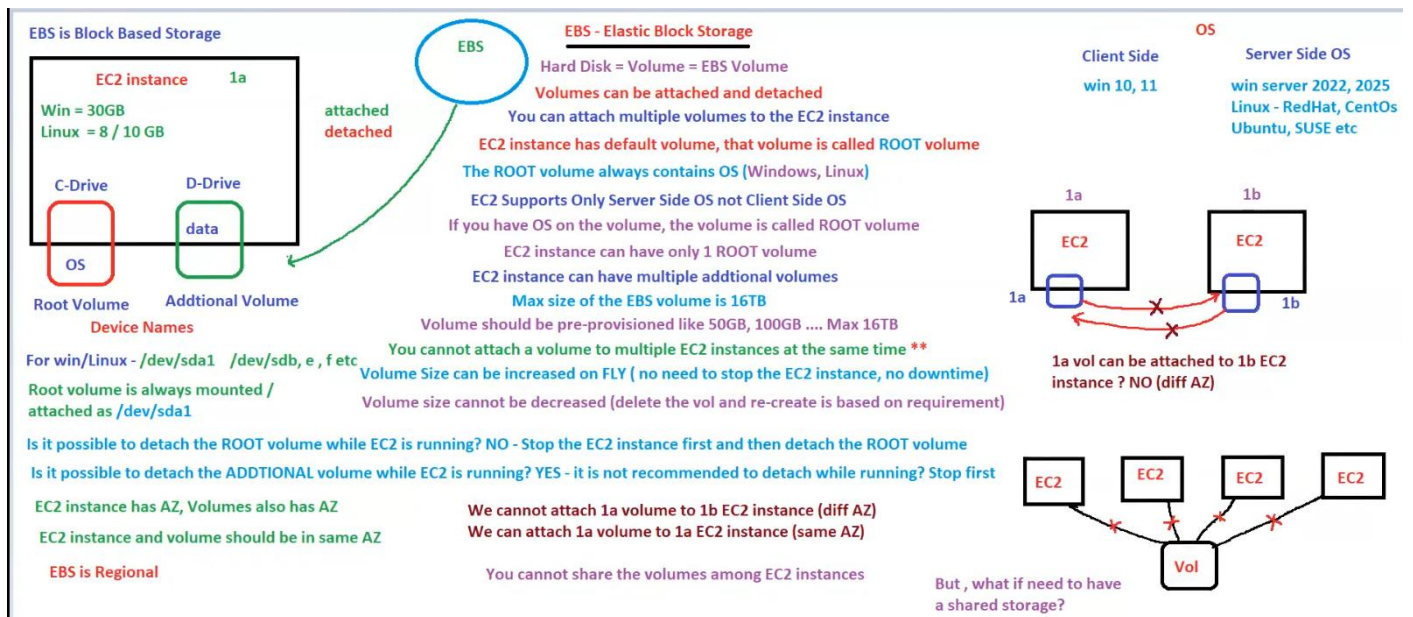


Schedule

9PM → STOPPED → STOPFn (Stop all the EC2 instances)

6AM → START → StartFn (Start all the EC2 instances)

AWS ELASTIC BLOCK STORAGE (EBS)



1. What is EBS (Elastic Block Storage)?

- EBS is a block-based storage system for EC2 instances.
- Acts like a hard disk (volume) that can be attached and detached from EC2 instances.
- EBS is Regional → Volumes exist within a specific AWS region.
- Each volume is tied to a single Availability Zone (AZ).

2. EC2 and EBS Volumes

- Every EC2 instance has a default volume, called the ROOT volume.

- The **Root Volume** always contains the OS (Windows/Linux).
 - EC2 instances **can have multiple additional volumes** for storing extra data.
-

3. Volume Types in EC2

- **Root Volume:**
 - Contains the **OS**.
 - **Always mounted at /dev/sda1** on Linux/Windows.
 - Cannot be detached while EC2 is running → **Instance must be stopped first**.
 - **Additional Volume:**
 - Used for **storing data**.
 - Can be detached while EC2 is running, but **not recommended**.
-

4. EBS Features & Limitations

- ✓ Supports only **Server-Side OS** (Windows Server, Linux).
 - ✓ **Max EBS Volume Size = 16TB**.
 - ✓ **EBS Volume can be increased on the fly** (No need to stop the EC2 instance).
 - ✗ **Cannot decrease volume size** (Need to create a new volume and migrate data).
 - ✗ **A volume cannot be attached to multiple EC2 instances at the same time**.
 - ✗ **An EBS volume is limited to a single AZ** (Cannot attach to EC2 in another AZ).
-

5. Attaching & Detaching Volumes

Action	Allowed?	Notes
Detach Root Volume while EC2 is running	✗ No	Stop EC2 first, then detach.
Detach Additional Volume while EC2 is running	✓ Yes	Not recommended, may cause data corruption.
Attach Volume to another EC2 in the same AZ	✓ Yes	Allowed if in the same AZ.
Attach Volume to an EC2 in a different AZ	✗ No	EBS is AZ-specific.

6. Shared Storage Between EC2 Instances?

- **EBS volumes cannot be shared across multiple EC2 instances.**
 - **Alternative:** Use **EFS (Elastic File System)** or **FSx** for shared storage.
-

7. Summary

Feature	Details
Root Volume	OS Storage, Mounted as /dev/sda1, Cannot detach while running.
Additional Volume	Extra storage, Can be attached/detached while running.
Max Volume Size	16TB
Resize Volume	✅ Increase without downtime, ❌ Cannot decrease.
Multi-Attach	❌ Cannot attach to multiple EC2 instances at the same time.
Cross-AZ Attachment	❌ EBS volume is restricted to a single AZ.
✅ Use EBS for block storage. ✅ For shared storage, use EFS or FSx.	

AWS S3 (SIMPLE STORAGE SERVICE)

In AWS, all services will start with
Simple and end with Service

SNS → Simple Notification Service
SES → Simple Email Service

S3 is a Object Based Storage

Laptop	S3
Folder	Bucket
Files	Objects
pushpa.mp4	KEY

S3 = Simple Storage Service

S3 is unlimited storage by AWS

S3 is used to store the files

S3 can store any kind of files (FLAT files)

With S3, we can upload, download, store the data and access your files

You cannot execute any files in S3

You cannot install, run, execute any files in S3

S3 is Serverless

AWS handles HA, Performance, Scalability etc for S3

Floppy → 2MB
CD's → 700MB
DVD's → 4.7GB
Pen Drives → 128GB
Hard Disks → 2TB

Bucket is a container of Objects

Object is a FILE

Name of the File / Object is a KEY

S3 is Regional

Buckets are Regional

S3 Supports Static WebSite hosting

Create a bucket and upload all the html files and
enable static website hosting

No need to worry about HA, performance, scalability etc because S3 handles it

1. What is AWS S3?

- S3 = Simple Storage Service.
- S3 provides unlimited storage managed by AWS.
- S3 is used for storing files (object-based storage).
- Supports any type of file (flat files like images, videos, documents, backups, etc.).
- S3 is Serverless – No need to manage infrastructure.
- AWS handles High Availability (HA), Performance, and Scalability.

2. How S3 Works? (Folder & File Concept)

Local System (Laptop) S3 Equivalent

Laptop	S3
Folder	Bucket
Files	Objects
File Name	Key

- **Bucket:** A container for storing objects (files).
 - **Object:** A stored file.
 - **Key:** The unique name assigned to an object (file name).
-

3. S3 Storage Features & Limitations

- ✅ Can upload, download, and store files.
 - ❌ Cannot execute, install, or run any files directly in S3.
 - ✅ Supports Static Website Hosting (for hosting HTML files).
 - ✅ S3 is Regional → Buckets exist within a specific AWS region.
 - ❌ Buckets are also Regional (Cannot access from different regions unless configured).
-

4. S3 Static Website Hosting

- S3 supports static website hosting.
- Steps to host a website on S3:
 1. Create a Bucket.
 2. Upload HTML files.
 3. Enable static website hosting.
 4. S3 will manage HA, Performance, and Scalability.

- ✅ No need for additional infrastructure like EC2.
-

5. Comparison of Traditional Storage vs S3

Storage Type Capacity

Floppy Disk	2MB
CD	700MB
DVD	4.7GB
Pen Drive	128GB

Storage Type Capacity

Hard Disk	2TB
S3	Unlimited 

6. Summary

Feature	Details
Type	Object Storage
Storage Limit	Unlimited 
Execution	Cannot execute files 
Static Website Hosting	Supported 
Region	S3 is Regional 
Bucket Region	Buckets are Regional 

-  Use S3 for file storage, backups, and hosting static websites.
-  For shared storage across multiple regions, enable S3 Cross-Region Replication.